
Basi

Operazioni

- Le quattro operazioni: +, *, -, /
- Esponenziazione: **

Divisione

```
3 / 4    # divisione normale → 0.75
3 // 4   # parte intera della divisione → 0
3 % 4    # resto della divisione → 3
```

Assegnamento

```
x = 5
```

Assegna il valore 5 alla variabile x (indipendentemente dal valore precedente di x).

Assegnamento composto

```
y = x + 3
```

Assegna alla variabile y il valore della variabile x in quel momento, aggiungendo 3 (indipendentemente dal valore precedente di y).

Funzioni built-in importanti

- `len(X)`: restituisce il numero di elementi in X.
 - `input()`: restituisce la stringa inserita dall'utente.
 - `type(X)`: restituisce il tipo di dato della variabile X.
 - `float(X)`: restituisce X come tipo float.
 - `int(X)`: restituisce X come tipo int.
 - `str(X)`: restituisce X come tipo stringa.
 - `round(X, <num>)`: restituisce X arrotondato a <num> decimali. Se <num> non è specificato, arrotonda all'intero più vicino.
 - `print(X)`: stampa X.
 - `help(X)`: restituisce la descrizione di X.
-

Decisioni, Cicli e Funzioni

Decisioni

In programmazione, le decisioni si implementano usando le **istruzioni condizionali**, che permettono al codice di seguire percorsi diversi a seconda di certe condizioni.

If semplice:

```
if <condizione>:  
    <istruzione> # NOTA: indentare con 4 spazi
```

If-else:

```
if <condizione>:  
    <istruzione>  
else:  
    <istruzione>
```

If-elif-else (catena di decisioni):

```
if <condizione>:  
    <istruzione>  
elif <condizione>:  
    <istruzione>  
else:  
    <istruzione>
```

Operatori condizionali:

- ==: uguale a
- <: minore di
- >: maggiore di
- !=: diverso da

Operatori logici:

- and: AND logico
- or: OR logico
- not: NOT logico

Esempio di condizione complessa:

```
if age > 18 and not is_student:  
    print("Adulto, non studente.")
```

Cicli

I cicli permettono l'esecuzione ripetuta di un blocco di codice.

For loop:

```
for <variabile> in <iterabile>:
```

<istruzione>

- <variabile> è la variabile del ciclo.
- <iterabile> è una collezione o un range predefinito.

range() è spesso usato per cicli numerici:

```
for i in range(5): # cicla da 0 a 4
    print(i)
```

While loop:

```
while <condizione>:
    <istruzione>
```

Il codice nel ciclo viene eseguito finché <condizione> è True:

```
counter = 0
while counter < 3:
    print(counter)
    counter += 1
```

Funzioni

Le funzioni sono blocchi di codice riutilizzabili progettati per svolgere compiti specifici.

Definizione di funzione:

```
def nome_funzione(parametri):
    <istruzione>
    return <valore> # opzionale
```

Esempio:

```
def sum(a, b):
    result = a + b
    return result
```

Chiamata di funzione:

```
total = sum(3, 4)
print(total) # Output: 7
```

Le funzioni possono accettare argomenti, elaborare dati e restituire risultati. L'uso corretto delle funzioni migliora modularità e riutilizzabilità del codice.

Stringhe

Le stringhe sono sequenze di caratteri, indicizzate a partire da 0; possono essere racchiuse tra doppi apici " o singoli ' e sono un tipo di dato immutabile.

```
A = "ciao"
```

Le stringhe sono **immutabili**, quindi non possono essere modificate una volta create, ma esistono operazioni e metodi per manipolarle e crearne di nuove.

Operazioni sulle stringhe

Lunghezza:

```
len(A) # numero di caratteri
```

Cambiamento del caso:

```
A.upper() # maiuscolo  
A.lower() # minuscolo
```

Esempio:

```
A = "Python"  
print(A.upper()) # PYTHON  
print(A.lower()) # python
```

Divisione in sottostringhe (split):

```
A.split(sep=B) # usa B come separatore
```

Se `sep` è omissso, si usa lo spazio bianco.

```
A = "apple,banana,cherry"  
print(A.split(",")) # ['apple', 'banana', 'cherry']
```

Concatenazione:

```
C = A + B # unisce due stringhe
```

```
A = "Hello"  
B = "World"  
C = A + " " + B  
print(C) # Hello World
```

Ripetizione:

```
C = A * n # ripete la stringa A n volte
```

```
A = "Hi"  
print(A * 3) # HiHiHi
```

Sostituzione:

```
A.replace(b, c) # sostituisce b con c
```

```
A = "banana"  
print(A.replace("a", "o")) # bonono
```

Ricerca:

```
A.find(b) # indice della prima occorrenza di b, -1 se non trovato
```

```
A = "Python"  
print(A.find("t")) # 2
```

```
print(A.find("z")) # -1
```

Conteggio:

```
A.count(b) # numero di occorrenze di b
```

```
A = "banana"
print(A.count("a")) # 3
```

Rimozione spazi:

```
A.strip() # rimuove spazi iniziali e finali
```

```
A = " Hello World "
print(A.strip()) # Hello World
```

Indicizzazione e slicing

Le stringhe sono indicizzate da 0:

```
A = "Hello"
print(A[0]) # H
print(A[-1]) # o
```

Slicing:

```
A[start:stop:step]
```

- `start`: indice iniziale (default 0)
- `stop`: indice finale (default fine stringa)
- `step`: intervallo tra caratteri (default 1)

```
A = "Hello"
print(A[1:4]) # ell
print(A[:2]) # Hl
print(A[::-1]) # olleH
```

Singoli caratteri e Unicode:

- `ord(c)`: dato un carattere, restituisce il suo codice Unicode
- `chr(n)`: dato un numero, restituisce il carattere corrispondente

```
print(ord('A')) # 65
print(ord('😬')) # 128578
```

Liste e Tuple

Liste

In Python, una lista è una sequenza di valori separati da virgole e racchiusi tra parentesi quadre `[]`. Le liste sono **mutabili**. Possono contenere elementi di qualsiasi tipo, comprese altre liste.

Esempi:

```
lista_vuota = []  
temperature = [24.3, 17.2, 18.2]  
nested_list = [1, [2, 3], 4]
```

Operazioni principali:

- Aggiungere: `A.append(X)`
- Rimuovere: `A.remove(X)`
- Invertire: `A.reverse()`
- Ordinare: `A.sort()`

```
A = [1, 2, 3]  
A.append(4)      # [1, 2, 3, 4]  
A.remove(2)      # [1, 3, 4]  
A.reverse()      # [4, 3, 1]  
A.sort()         # [1, 3, 4]
```

Funzioni comuni e slicing:

```
len(A)           # lunghezza  
A[start:stop:step] # slicing
```

```
A = [0,1,2,3,4,5]  
print(A[1:4])    # [1,2,3]  
print(A[::-2])   # [0,2,4]  
print(A[::-1])   # [5,4,3,2,1,0]
```

Tuple

Una **tuple** è simile a una lista ma **immutabile**. Non può essere modificata dopo la creazione.

Si usa la sintassi

con parentesi tonde ().

Esempi:

```
origine = (0, 0)  
point = (2.5, 4.3)  
mixed_tuple = (1, "hello", [3,4])
```

Operazioni sulle tuple:

- Supportano molte operazioni delle liste eccetto quelle che modificano il contenuto (append, remove, ecc.)
- `len(T)` per lunghezza
- Slicing: `T[start:stop:step]`

```
T = (0,1,2,3,4,5)  
print(T[1:4])    # (1,2,3)
```

Perfetto, ti traduco tutto in italiano mantenendo la stessa struttura schematica:

Insiemi (Sets)

Un **set** in Python rappresenta una collezione di elementi **unici** e **non ordinati**.

Caratteristiche di un Set:

- Gli insiemi si definiscono con le parentesi graffe {}.
- Gli elementi in un set devono essere **immutabili** (ad esempio: numeri, stringhe, tuple).
- I set sono utili per operazioni come **unione**, **intersezione** e **differenza**.

Esempi:

```
primi_umani = {"Adam", "Eve"}      # Un set di persone
primi_figli = {"Abel", "Kaine"}    # Un altro set
```

Operazioni sugli Insiemi:

- **Unione (A | B)** → restituisce l'unione di A e B:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B) # Output: {1, 2, 3, 4, 5}
```

- **Intersezione (A & B)** → restituisce gli elementi in comune:

```
print(A & B) # Output: {3}
```

- **Differenza (A - B)** → restituisce gli elementi in A ma non in B:

```
print(A - B) # Output: {1, 2}
```

Modifica di un Set:

- **Aggiungere un elemento** → `A.add(X)`

```
A = {1, 2}
A.add(3)
print(A) # Output: {1, 2, 3}
```

- **Rimuovere un elemento** → `A.discard(X)`
(se l'elemento non esiste, non dà errore)

```
A.discard(2)
print(A) # Output: {1, 3}
```

Dizionari (Dictionaries)

Un **dizionario** in Python è una collezione di **coppie chiave-valore**, dove:

- Ogni **chiave** è **unica** e **immutabile** (numeri, stringhe, tuple).

- I **valori** possono essere di qualsiasi tipo e possono ripetersi.

Caratteristiche di un Dizionario:

- I dizionari si definiscono con {}.
- Le chiavi e i valori sono separati da :.
- Sono **non ordinati** fino a Python 3.6, mentre da Python 3.7 in poi mantengono l'ordine di inserimento.

Esempi:

```
dizionario_vuoto = {}
```

```
to_buy = {"Biscotti": 3, "Birra": 4, "Pasta": 8}  
print(to_buy["Biscotti"]) # Output: 3
```

Iterare in un Dizionario:

- **Sulle chiavi:**

```
for item in to_buy:  
    print(item)  
# Output:  
# Biscotti  
# Birra  
# Pasta
```

- **Sui valori:**

```
for value in to_buy.values():  
    print(value)  
# Output:  
# 3  
# 4  
# 8
```

- **Sulle coppie chiave-valore:**

```
for key, value in to_buy.items():  
    print(f"{key}: {value}")  
# Output:  
# Biscotti: 3  
# Birra: 4  
# Pasta: 8
```

Differenze principali tra Set e Dizionario:

- Un **set** è una collezione di elementi unici e non ordinati.
 - Un **dizionario** è una collezione di **coppie chiave-valore**.
 - I set non hanno chiavi o valori, i dizionari sì.
 - Nei set, gli elementi devono essere immutabili; nei dizionari, solo le chiavi devono esserlo.
-

Lavorare con i file

Schema della sezione

Python fornisce funzioni integrate per lavorare con i file in modo efficiente. I file possono essere aperti, letti, scritti e chiusi usando la funzione `open()` di Python.

Aprire un file

Si usa la funzione `open()` per aprire un file:

```
file = open("example.txt", "r")    # Apertura in lettura (modalità predefinita)
file = open("example.txt", "w")    # Apertura in scrittura (sovrascrive il
contenuto esistente)
file = open("example.txt", "a")    # Apertura in modalità append (aggiunge al
contenuto)
file = open("example.txt", "rb")   # Apertura in modalità binaria
```

Leggere da un file

Sono disponibili diversi metodi per leggere da un file:

```
file = open("example.txt", "r")
content = file.read()           # Legge l'intero contenuto
line = file.readline()          # Legge una sola riga
lines = file.readlines()        # Legge tutte le righe in una lista
file.close()                    # Chiudere sempre il file
```

Usare `with` garantisce una corretta gestione delle risorse:

```
with open("example.txt", "r") as file:
    content = file.read() # Il file viene chiuso automaticamente
```

Scrivere in un file

Per scrivere dati, si usa la modalità `w` (write) o `a` (append):

```
with open("example.txt", "w") as file:
    file.write("Hello, World!\n")
```

Aggiungere contenuto:

```
with open("example.txt", "a") as file:
    file.write("Appending new content.\n")
```

Lavorare con file binari

Per gestire file binari come immagini:

```
with open("image.png", "rb") as file:
    data = file.read()
```

Verificare se un file esiste

Si può usare il modulo `os` o `pathlib`:

```
import os
if os.path.exists("example.txt"):
    print("Il file esiste")
```

Con `pathlib`:

```
from pathlib import Path
if Path("example.txt").exists():
    print("Il file esiste")
```

Importare una funzione (o una variabile) da un file

Se abbiamo una funzione `myFunction` definita in un file `myfile.py`:

```
from myfile import myFunction
```

NumPy

Schema della sezione

NumPy è una libreria fondamentale su cui si basano la maggior parte delle librerie Python per l'elaborazione dei dati. Aggiunge il supporto per array e matrici multidimensionali.

La documentazione ufficiale è disponibile qui: <https://numpy.org/doc/stable/>

⚠️ **NOTA:** NumPy **non è incluso** nell'installazione predefinita di Python; deve essere installato manualmente.

```
pip install --upgrade pip
```

```
python3 -m pip install -U numpy
```

NumPy fornisce:

- ✓ Array multidimensionali efficienti
- ✓ Operazioni matematiche avanzate
- ✓ Gestione semplice dei file numerici
 - Indicizzazione flessibile e manipolazione di dati multidimensionali arbitrari.
 - Molte routine numeriche (algebra lineare, analisi di Fourier, ottimizzazione globale, ecc.) con vari algoritmi.
 - Supporto per la generazione di numeri casuali da un'ampia gamma di distribuzioni.

- Un'interfaccia Python per molte librerie sottostanti con implementazioni efficienti di algoritmi numerici.

Importare NumPy

```
from numpy import *  
import numpy as np
```

Alcuni sottomoduli devono essere importati separatamente. Ad esempio, il modulo di algebra lineare **linalg**:

```
import numpy.linalg as la
```

Controllare la versione di NumPy

```
import numpy  
print(numpy.__version__)
```

Esempio di output:

1.24.3

Il concetto centrale di NumPy: l'array N-dimensionale

Il concetto chiave di NumPy è l'**array n-dimensionale**, chiamato anche **tensore**. In NumPy è rappresentato dall'oggetto **ndarray**.

- I vettori sono array 1D.
- Le matrici sono array 2D.

Per qualsiasi array è possibile controllare dimensionalità e forma con gli attributi **shape**, **ndim**, e **dtype**:

```
a = numpy.zeros(10)  
print(a)  # array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])  
  
print(a.shape)  # (10,)  
print(a.ndim)   # 1  
print(a.dtype)  # dtype('float64')
```

Creazione di array da liste e tuple

Diversamente da liste e tuple Python, gli array NumPy contengono elementi di un unico tipo di dato.

```
a = numpy.array([[1, 2, 3], [4, 5, 6]])  
b = numpy.array([[10, 11, 12], [13, 14, 15]])
```

Operazioni di base sugli array

- **Somma tra array (+):**

```
a + b
```

```
# Risultato:
```

```
array([[11, 13, 15],  
       [17, 19, 21]])
```

- **Moltiplicazione per scalare (*):**

```
c = 2 * a
```

```
print(c)
```

```
# Risultato:
```

```
array([[ 2,  4,  6],  
       [ 8, 10, 12]])
```

- **Reshape (cambio forma):**

```
d = numpy.array([x for x in range(27)])
```

```
print(d.reshape(3, 3, 3))
```

Conversione di tipo (astype)

```
I = numpy.eye(3)
```

```
print(I.astype('float64'))
```

```
print(I.astype(bool))
```

Stacking di array

- **Orizzontale (hstack):**

```
numpy.hstack((a, b))
```

- **Verticale (vstack):**

```
numpy.vstack((a, b))
```

Trasposizione e moltiplicazione di matrici

```
b.T          # Trasposizione
```

```
a @ b.T      # Prodotto matriciale
```

```
numpy.matmul(a, b.T)
```

Trovare indici con where

```
x = numpy.array([1, 2, 3, 4, 5])
```

```
y = numpy.array([1, 3, 2, 4, 5])
```

```
numpy.where(x == y)
# Risultato: (array([0, 3, 4]),)
```

Lavorare con file e ndarray

- **Salvataggio con tofile:**
- **Caricamento con fromfile:**
- **Metodi migliori: savetxt e loadtxt**

```
numpy.savetxt("data.dat", xx)
yy = numpy.loadtxt("data.dat")
```

Generazione veloce di array

- **Sequenze (arange):**

```
np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
```

- **Numeri equidistanti (linspace):**

```
np.linspace(0, 1, 5) # [0, 0.25, 0.5, 0.75, 1]
```

- **Array casuali (random):**

```
np.random.rand(3, 3)
```

Maschere booleane e selezione condizionale

```
arr = np.array([10, 15, 22, 5, 30])
print(arr[arr > 10]) # [15, 22, 30]
```

Sostituzione valori:

```
arr[arr < 20] = 0
```

Min, Max, Somma, Media, Mediana, Deviazione standard

```
arr.min()
arr.max()
arr.sum()
np.mean(arr)
np.median(arr)
np.std(arr)
```

Ordinamento e ricerca

```
np.sort(arr)          # Ordina
arr.argmax()          # Indice del max
```

```
arr.argmax()      # Indice del min
```

Concatenazione e suddivisione

```
np.concatenate((a, b))  # Unisce array
np.split(arr, 2)        # Divide in due parti
```

Operazioni elemento per elemento

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr * 10)
```

Array multidimensionali

Selezione:

```
print(arr[0, :])  # Prima riga
print(arr[:, 1])  # Seconda colonna
```

Flatten:

```
print(arr.flatten())  # Da 2D a 1D
```

Programmazione Orientata agli Oggetti (OOP) in Python

Python supporta sia la programmazione procedurale/funzionale sia la programmazione orientata agli oggetti (OOP), permettendo flessibilità nello sviluppo.

Modello Object Factory

Python segue un modello di **object-factory**, dove ogni classe viene utilizzata per creare nuovi oggetti (**istanze**).

Ereditarietà

L'ereditarietà permette di creare nuove classi a partire da classi esistenti, estendendole con funzionalità aggiuntive.

Dichiarazione di una Classe in Python

Vecchio Stile (Python 2.x)

```
class SistemaComplesso:
    pass
```

```
class SistemaComplesso():
    pass
```

```
oggetto = SistemaComplesso()
```

Nuovo Stile (Python 3.x)

```
class SistemaComplesso(object):  
    pass
```

```
oggetto = SistemaComplesso()
```

Nota: Tutte le classi ereditano implicitamente da `object`.

Documentare una Classe

Una classe può avere una docstring opzionale che può essere visualizzata usando la funzione `help()`.

```
class SistemaComplesso(object):  
    """  
    Questa describe la classe quando si usa help.  
    """  
    pass
```

```
help(SistemaComplesso)
```

Output:

```
Help on class SistemaComplesso in module __main__:
```

```
class SistemaComplesso(builtins.object)  
| Questa describe la classe quando si usa help.  
|  
| Data descriptors defined here:  
|  
| __dict__  
|     Dictionary for instance variables (if defined)  
|  
| __weakref__  
|     List of weak references to the object (if defined)
```

Creazione e Inizializzazione di Oggetti

In Python, la **creazione** e l'**inizializzazione** di un oggetto avvengono in due fasi separate, usando `__new__` e `__init__`.

Metodo `__new__`

- Crea e ritorna una nuova istanza non inizializzata della classe.
- Viene chiamato automaticamente durante la creazione dell'oggetto.
- Se non sovrascritto, viene usato il metodo di default ereditato da `object`.

Metodo `__init__`

- Inizializza la nuova istanza creata.
- Chiamato subito dopo `__new__`.
- Viene solitamente usato per assegnare attributi all'istanza.

```
class SistemaComplesso(object):  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

Nota: La funzione `__init__` deve ritornare `None`; altrimenti viene sollevata un'eccezione.

Distruzione dell'Oggetto: `__del__`

- Il metodo `__del__` viene chiamato quando un oggetto viene distrutto.
- È l'equivalente di un **distruttore** in C++.
- A causa del garbage collector di Python, viene eseguito solo quando tutte le referenze all'oggetto vengono rimosse.

```
class Demo:  
    def __del__(self):  
        print("Oggetto distrutto")
```

Ereditarietà

Python supporta sia l'**ereditarietà singola** sia **multipla**.

Ereditarietà Singola

```
class B(object): pass  
class A(B): pass
```

Ereditarietà Multipla

```
class B(object): pass  
class C(object): pass  
class D(object): pass  
class A(B, C, D): pass
```


- L'ereditarietà multipla può generare gerarchie complesse, dove determinare l'ordine di risoluzione dei metodi (**MRO**) non è banale.
- Python risolve ambiguità usando l'algoritmo **C3 Linearization**.
- L'attributo `__mro__` mostra l'ordine di risoluzione dei metodi:

```
print(A.__mro__)
```

Metodo `super()`

La funzione `super()` aiuta a chiamare un metodo della **classe genitore**.

- Utile soprattutto nell'ereditarietà multipla per evitare chiamate ridondanti (problema del diamante).

```
class Parent:
    def greet(self):
        print("Ciao dal Parent")
```

```
class Child(Parent):
    def greet(self):
        super().greet()
        print("Ciao dal Child")
```

```
c = Child()
c.greet()
```

Output:

Ciao dal Parent

Ciao dal Child

Visibilità degli Attributi

- Per default, tutti gli attributi in Python sono **pubblici**.
- È possibile rendere un attributo **privato** usando convenzioni di denominazione.

Attributi Privati

- Prefix con `__` (doppio underscore) rende l'attributo privato.

```
class Example:
    def __init__(self):
        self.__private_attr = "Nascosto"
```

Attributi Speciali

- Gli attributi che iniziano e finiscono con `__` hanno significati speciali.
- Esempi:
 - `__class__` memorizza un riferimento alla classe dell'oggetto.
 - `__name__` memorizza il nome della classe.

```
obj = Example()
```

```
print(obj.__class__)
```

Ecco la traduzione in italiano:

Matplotlib

Schema della sezione

Grafici con Matplotlib

La libreria **Matplotlib** permette di disegnare grafici 2D in vari modi.

Le quantità da plottare sono **ndarray di numpy**, ma anche una **lista** viene automaticamente convertita in array quando viene passata come argomento alla funzione `plot`.

Esempio base:

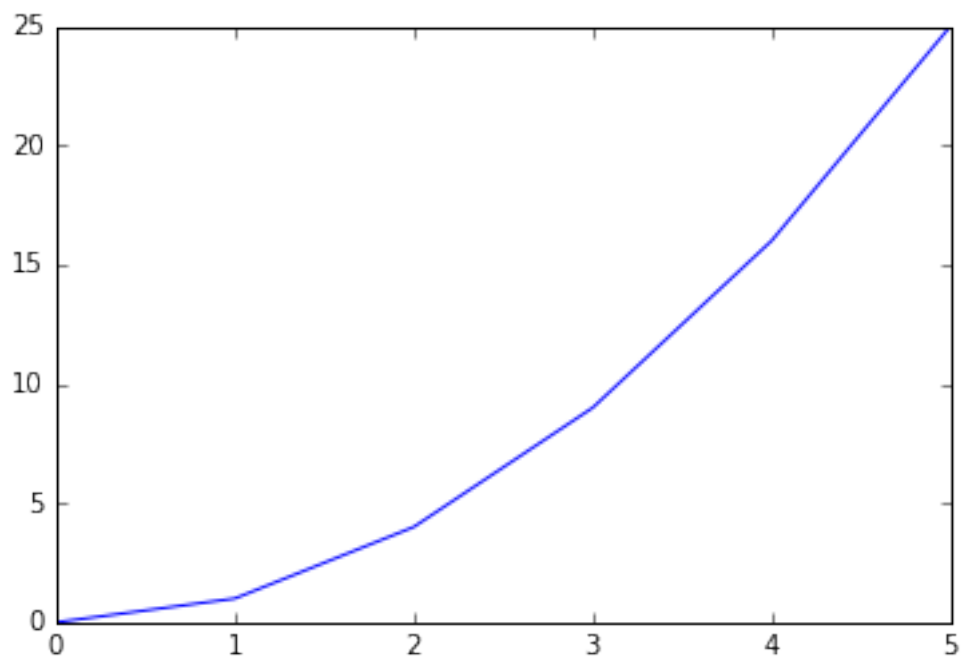
```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
plt.plot([0.0, 1.0, 4.0, 9.0, 16.0, 25.0])
```

```
plt.show()
```

- Una **singola lista** è interpretata come valori sull'asse **y**, mentre i valori sull'asse **x** vengono considerati i loro indici.
- Se vengono passate **due liste**, la prima è interpretata come valori dell'asse **x**.
- Una **stringa come terzo parametro** è interpretata come descrizione del colore, dello stile della linea, ecc.



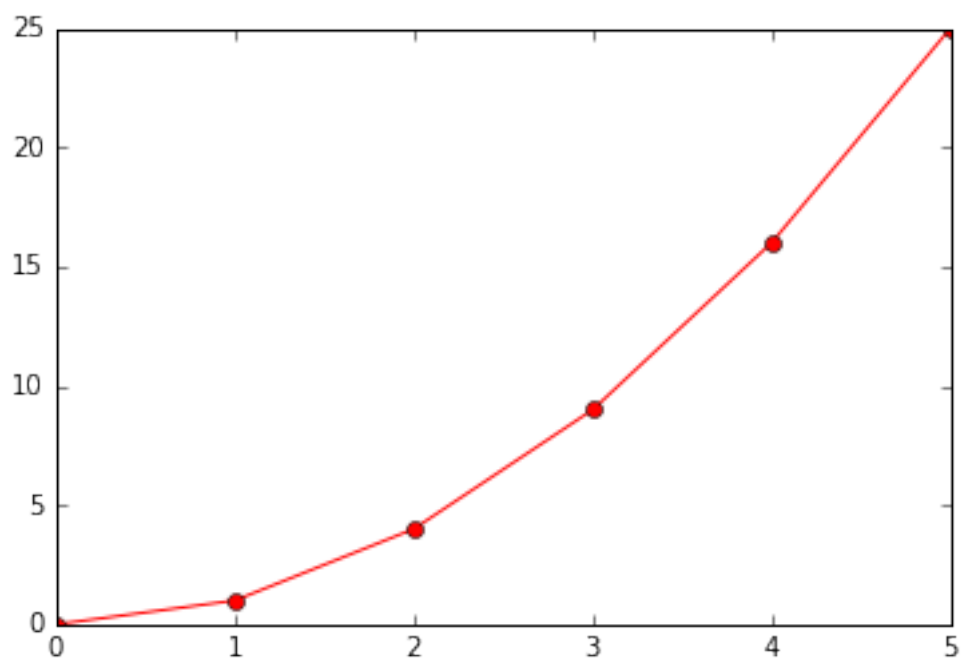
Esempio: linea continua (parametro '-') + punti rossi (parametro 'or')

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
plt.plot([0.0, 1.0, 2.0, 3.0, 4.0, 5.0],  
         [0.0, 1.0, 4.0, 9.0, 16.0, 25.0],  
         'or-')
```

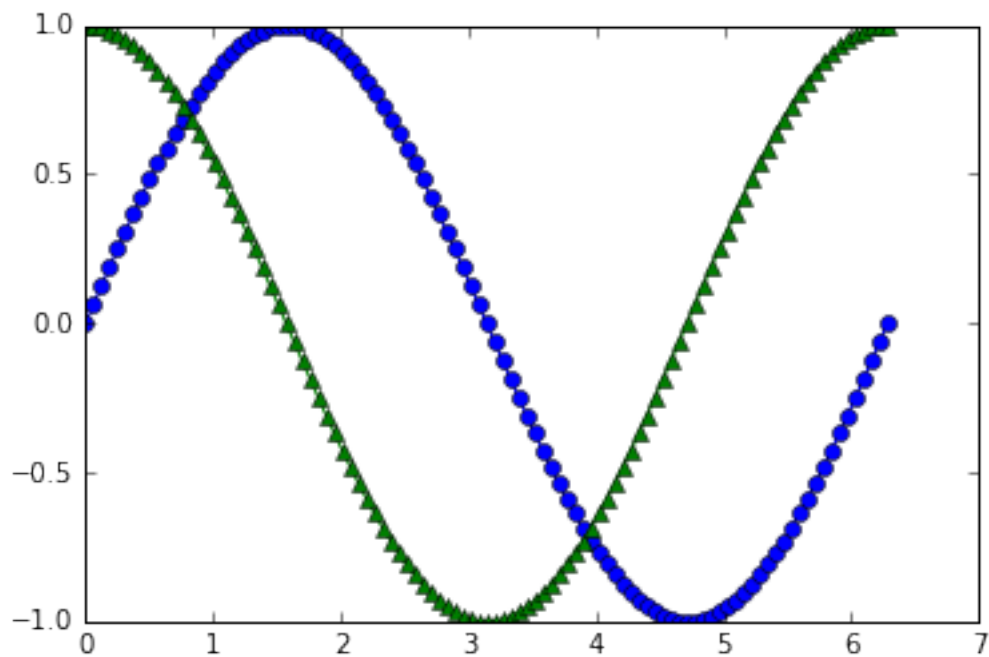
```
plt.show()
```



Si possono plottare più ndarray contemporaneamente:

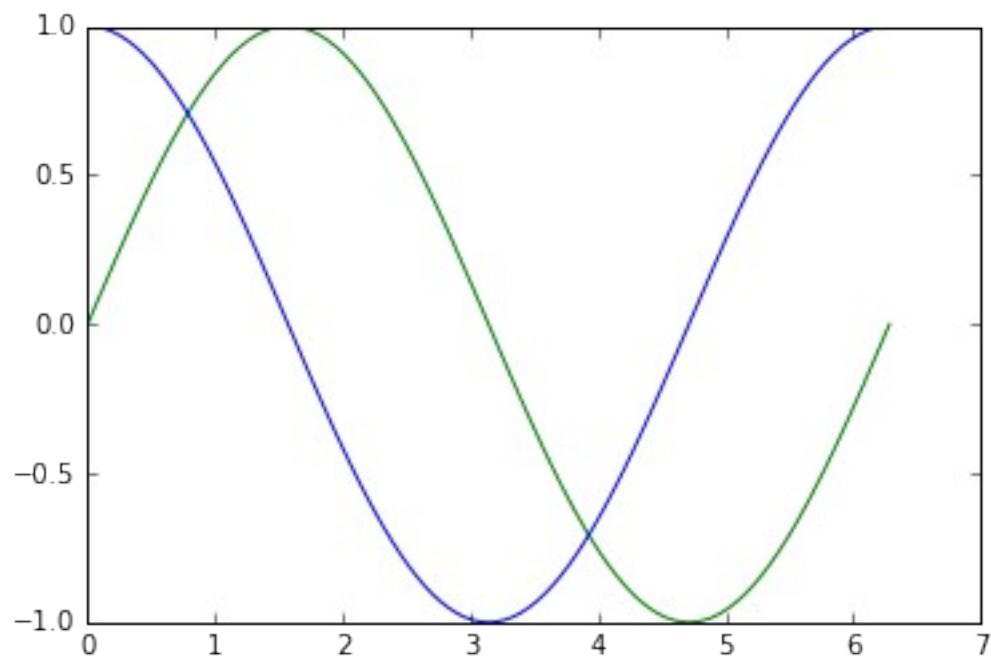
```
x = np.linspace(0.0, 2.0*np.pi, 101)
y = np.sin(x)
z = np.cos(x)
```

```
plt.plot(x, y, 'o-')
plt.plot(x, z, '^--')
plt.show()
```



Si possono plottare più funzioni con un singolo comando (nota i simboli dei colori):

```
plt.plot(x, y, 'g-', x, z, 'b-')
plt.show()
```



Opzioni di colore e stile delle linee

Il parametro opzionale (terzo argomento), che di default è una linea blu se non specificato, permette di indicare **colore**, **marker** e **stile della linea** in forma compatta.

Colori più comuni:

- b : blu
- g : verde
- r : rosso
- c : ciano
- m : magenta
- y : giallo
- k : nero
- w : bianco

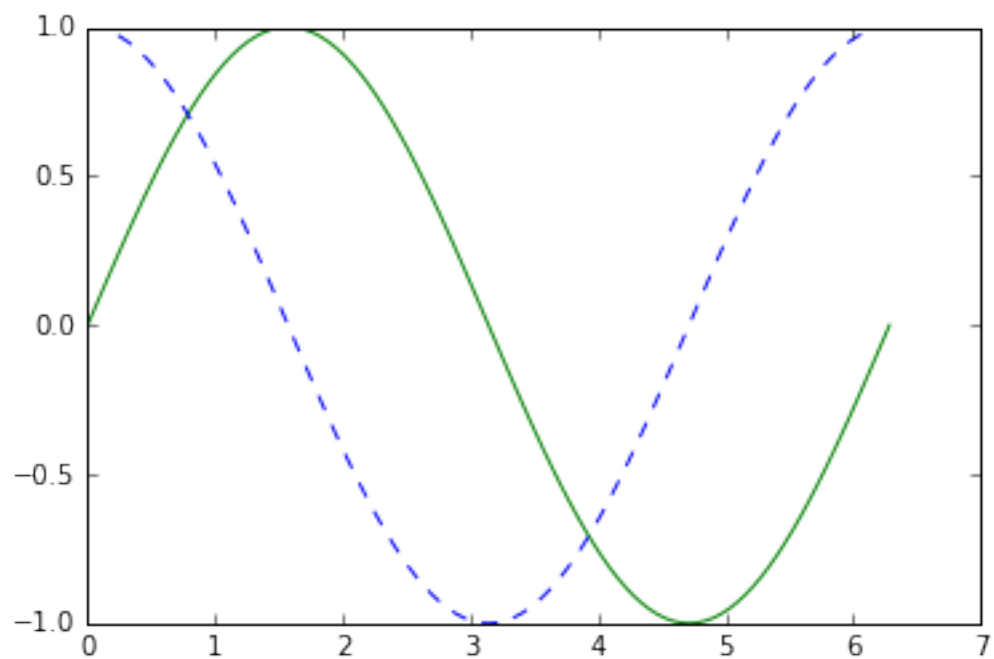
Stili di linea e marker:

- - : linea continua
- - - : linea tratteggiata
- - . : linea punto-tratto
- : : linea puntinata
- . : marker a punto
- , : marker a pixel

- `o` : marker a cerchio
- `v` : triangolo verso il basso
- `^` : triangolo verso l'alto
- `<` : triangolo verso sinistra
- `>` : triangolo verso destra
- `s` : quadrato
- `p` : pentagono
- `*` : stella
- `h` : esagono
- `+` : più
- `x` : x
- `D` : diamante
- `|` : linea verticale
- `_` : linea orizzontale

Esempi:

```
plt.plot(x, y, 'g-', x, z, 'b--')  
plt.show()
```



Gestione degli errori

Schema della sezione

Python fornisce un meccanismo integrato per gestire gli errori a runtime tramite la costruzione **try-except**.

```
try:
    # Codice che può sollevare un'eccezione
    risky_operation()
```

```
except ExceptionType:
    # Codice per gestire l'eccezione
    handle_error()
```

- Il blocco **try** contiene codice che potrebbe sollevare un'eccezione.
- Il blocco **except** intercetta l'eccezione ed esegue codice alternativo per gestirla.
- **ExceptionType** specifica quale eccezione catturare (es. `ValueError`, `ZeroDivisionError`).

Esempio: gestione della divisione per zero

```
try:
    num = int(input("Inserisci un numero: "))
    result = 10 / num
    print("Risultato:", result)
except ZeroDivisionError:
    print("Errore: non è consentita la divisione per zero.")
```

Se l'utente inserisce 0, Python solleva una `ZeroDivisionError`, che viene catturata e gestita. Il programma **non va in crash**, ma stampa un messaggio di errore.

Catturare eccezioni multiple

```
try:
    num = int(input("Inserisci un numero: "))
    result = 10 / num
    print("Risultato:", result)
except ZeroDivisionError:
    print("Errore: impossibile dividere per zero.")
except ValueError:
    print("Errore: inserisci un numero valido.")
```

- Se l'utente inserisce 0, il programma cattura `ZeroDivisionError`.
 - Se l'utente inserisce un valore non numerico, cattura `ValueError`.
-

Catturare tutte le eccezioni

```
try:
    risky_operation()
except Exception as e:
    print("Si è verificato un errore:", e)
```

- `Exception` è la classe base di tutte le eccezioni integrate.
 - `as e` memorizza il messaggio di errore, utile per il debug.
 - **Attenzione:** catturare tutte le eccezioni può nascondere bug.
-

Uso di `else` con `try-except`

```
try:
    num = int(input("Inserisci un numero: "))
    result = 10 / num
except ZeroDivisionError:
    print("Non puoi dividere per zero!")
else:
    print("Successo! Il risultato è", result)
```

- Il blocco **`else`** viene eseguito **solo se** il blocco `try` non solleva eccezioni.
 - Se `num = 2`, il programma stampa:
Successo! Il risultato è 5.0.
-

Uso di `finally` per liberare risorse

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("Errore: file non trovato.")
finally:
    print("Chiusura del file.")
    file.close()
```

- Il blocco **`finally`** viene sempre eseguito, con o senza eccezioni.
- Utile per la gestione delle risorse (es. chiudere file, liberare connessioni al database).

Sollevare eccezioni manualmente

```
def check_age(age):  
    if age < 18:  
        raise ValueError("L'età deve essere almeno 18.")  
    return "Accesso consentito."
```

```
try:  
    print(check_age(16))  
except ValueError as e:  
    print("Errore:", e)
```

- L'istruzione **raise** genera un errore personalizzato se `age < 18`.
- Il blocco `except` cattura e stampa il messaggio di errore.

Pacchetto Pandas: DataFrame (e Series)

Schema della sezione

Pandas fornisce due oggetti principali per la gestione dei dati:

- **Series** – Un array monodimensionale etichettato, capace di contenere dati di qualsiasi tipo.
- **DataFrame** – Una struttura dati bidimensionale etichettata, simile a una tabella con righe e colonne.

Creare una Series

Una **Series** in Pandas è un array monodimensionale con un indice. Può contenere interi, float, stringhe e anche valori mancanti (NaN).

```
import pandas as pd  
import numpy as np  
  
s = pd.Series([1, 3, 5, np.nan, 6, 8])  
print(s)
```

Output:

```
0    1.0  
1    3.0  
2    5.0  
3    NaN
```

```
4      6.0
5      8.0
dtype: float64
```

Creare un DataFrame

Un **DataFrame** è una struttura dati bidimensionale che somiglia a una tabella. È costituita da righe e colonne, ognuna con etichette (indici).

DataFrame con dati casuali

```
dates = pd.date_range("20130101", periods=6) # Crea un intervallo di date
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))

print(df)
```

Output:

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
...				

DataFrame con diversi tipi di dati

```
df2 = pd.DataFrame(
    {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)), dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
)

print(df2)
```

Output:

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo

```
1  1.0 2013-01-02  1.0  3  train  foo
2  1.0 2013-01-02  1.0  3   test  foo
3  1.0 2013-01-02  1.0  3  train  foo
```

Accesso ai dati in un DataFrame

- **Prime righe**

```
df.head()
```

- **Ultime 3 righe**

```
df.tail(3)
```

- **Indici e nomi delle colonne**

```
print(df.index)
```

```
print(df.columns)
```

- **Convertire in NumPy array**

```
df.to_numpy()
```

(Nota: `to_numpy()` converte il DataFrame in un array NumPy, ma perde le etichette delle colonne.)

Statistiche

Per riassumere rapidamente i dati numerici:

```
df.describe()
```

Output (esempio):

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.073711	-0.431125	-0.687758	-0.233103
std	0.843157	0.922818	0.779887	0.973118
...				

Ordinamento

- **Per indice**

```
df.sort_index(axis=1, ascending=False)
```

- **Per valori di una colonna**

```
df.sort_values(by="B")
```

Selezione

- **Colonna singola (ritorna una Series)**

```
df["A"]
```

- **Selezione righe per intervallo di indici**

```
df[0:3]
```

```
df["20130102":"20130104"]
```

- **Selezione righe e colonne specifiche**

```
df.loc[:, ["A", "B"]]
```

```
df.loc["20130102":"20130104", ["A", "B"]]
```

- **Selezione con condizioni**

```
df[df > 0]
```

(restituisce solo valori positivi, gli altri diventano NaN)

Grafici (-> Matplotlib)

Pandas si integra bene con **Matplotlib** per la visualizzazione dei dati.

```
import matplotlib.pyplot as plt
```

```
ts = pd.Series(np.random.randn(1000), index=pd.date_range("1/1/2000",  
periods=1000))
```

```
ts = ts.cumsum()
```

```
ts.plot()
```

```
plt.show()
```

Se usi **Jupyter Notebook**, basta chiamare `ts.plot()` per visualizzare il grafico.

Per plottare tutte le colonne di un DataFrame:

```
df = pd.DataFrame(np.random.randn(1000, 4), index=ts.index, columns=["A", "B",  
"C", "D"])
```

```
df = df.cumsum()
```

```
df.plot()
```

```
plt.show()
```

Importazione ed Esportazione Dati

- **CSV**

```
df.to_csv("output.csv")          # Salva in CSV
pd.read_csv("output.csv")        # Carica da CSV
```

- **Excel**

```
df.to_excel("output.xlsx", sheet_name="Sheet1")  # Salva in Excel
pd.read_excel("output.xlsx", "Sheet1")          # Carica da Excel
```

Riepilogo veloce (per i pigri)

Operazione	Metodo
Creare una Series	<code>pd.Series(...)</code>
Creare un DataFrame	<code>pd.DataFrame({...})</code>
Prime/Ultime righe	<code>df.head(), df.tail()</code>
Statistiche	<code>df.describe()</code>
Ordinamento per indice	<code>df.sort_index()</code>
Ordinamento per valori	<code>df.sort_values(by="B")</code>
Selezione colonna	<code>df["A"]</code>
Selezione righe	<code>df.loc[...], df.iloc[...]</code>
Filtrare dati	<code>df[df > 0]</code>
Grafici	<code>df.plot()</code>
Leggere/Scrivere CSV	<code>df.to_csv(), pd.read_csv()</code>
Leggere/Scrivere Excel	<code>df.to_excel(), pd.read_excel()</code>
